DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

The Stack Is Your Friend

CMSC 197 GDD Case Study 2

Prepared by: Rene Andre B. Jocsing

Published: March 10, 2026

Case Study Objectives

- Understand why flat FSMs cannot model SRPG game flow
- Analyze the pushdown automaton as applied to a live engine
- Trace the state lifecycle and deferred transition system
- Adapt the architecture to Godot using `RefCounted`-based states

Background

Lex Talionis is an open-source engine for building turn-based strategy RPGs in the *Fire Emblem* style (see the Component System case study for a deeper introduction to the engine).

In Lex Talionis, the **State System** handles *what happens*—the entire flow of the game, from the title screen to combat to cutscenes to menus. Every SRPG has a complex lifecycle: the player starts at a title screen, loads a save, enters a level, selects a unit, picks a movement destination, opens an action menu, chooses to attack, selects a target, watches combat play out, sees experience awarded, possibly a level-up screen, and then control returns to the map—or maybe the unit has Canto and can move again after attacking.

That is just one turn for one unit. The State System is the architectural backbone that makes all of it manageable.

Context: Why a Flat FSM Fails

If you have done any game programming, you have encountered the **Finite State Machine** (FSM) pattern: the game is always in exactly one state at a time, and inputs cause transitions between states. Simple enough for a platformer (Idle → Running → Jumping → Falling).

An SRPG is not simple. Consider this gameplay sequence:

```
Title Screen -> Load Save -> Level Start -> Player Phase
  -> Select Unit -> Move Unit -> Action Menu -> Attack
    -> Select Target -> Combat Animation -> EXP Screen
      -> Level Up -> Return to Map -> (Canto?) Move Again
        -> Wait -> Next Unit or End Turn
-> Enemy Phase -> AI Think -> Enemy Move -> Enemy Attack
  -> Combat -> (Unit dies?) Death Animation -> Continue AI
-> Next Turn -> Event Cutscene -> ...
```

A flat FSM cannot express this. You need states that can *nest*—an event cutscene can trigger mid-combat, a menu can open on top of the map, combat can interrupt the player's turn flow. What you really need is a **pushdown automaton**: a state machine with a *stack*.

The Problem

The Spaghetti Problem

Your first instinct for game flow might be a big switch statement in the main loop:

```
while running:
    if mode == "title":
        handle_title_input()
        draw_title()
    elif mode == "map":
        handle_map_input()
        draw_map()
    elif mode == "menu":
        handle_menu_input()
        draw_menu()
    elif mode == "combat":
        handle_combat_input()
        draw_combat()
    # ... 30 more elif branches
```

This falls apart immediately. Control flow is implicit—hidden in `mode = "something_else"` assignments scattered everywhere. Adding a new mode means touching the main loop. There is no clean way to handle "draw the map *behind* the menu" or "pause the current state while an event plays."

The Deep Nesting Problem

Embedding flow in nested function calls gives you clear sequencing but terrible flexibility:

```
def player_turn():
    unit = select_unit()
    dest = pick_move(unit)
    move_unit(unit, dest)
    action = show_menu(unit)
    if action == "attack":
        target = pick_target(unit)
        run_combat(unit, target)
        if unit.has_canto:
            player_turn()  # Recursion!
```

What if `run_combat` needs to show a cutscene mid-fight? What if `show_menu` needs to open a submenu? You end up passing callbacks and coroutines everywhere, and the call stack becomes the implicit game state—good luck saving and loading *that*.

The State Explosion Problem

Even with a proper FSM, an SRPG has an absurd number of states. Lex Talionis registers **120+ distinct states**. You need a pattern that makes each state self-contained, composable, and easy to navigate between.

Why This Approach?

Lex Talionis uses a **stack-based state machine** (pushdown automaton). The core ideas:

Design Principles

1. **States are objects**: each game mode is a class with a standard lifecycle (`start`, `begin`, `take_input`, `update`, `draw`, `end`, `finish`); state logic is encapsulated, not spread across a switch statement
2. **Stack, not graph**: states are pushed and popped, not transitioned between in a flat graph; "open menu on top of map" is pushing `MenuState`; "close menu" is popping it; the map state is still there underneath, untouched
3. **Deferred transitions**: state changes are *queued*, not immediate; when a state calls `game.state.change('comba` the transition happens at end-of-frame after the current state finishes its update cycle, preventing re-entrancy bugs
4. **Transparent layering**: states declare whether they are `transparent`; the machine walks down the stack until it finds a non-transparent state, then draws upward; a transparent `EventState` renders on top of the map without the map knowing about events
5. **Memory bus**: states communicate through `game.memory`, a shared dictionary; a state does not need to import or reference the state that will read its data
6. **Serializable**: the entire state stack is a list of state names; saving is `[state.name for state in self.state]`; loading re-instantiates from the registry

---

Architecture Deep Dive

The Base State

Everything starts here:

```
class State():
    name: ClassVar[str] = None  # Unique identifier
    in_level = True             # Is this state part of gameplay?
    show_map = True             # Should the map be visible?
    transparent = False         # Draw states below on the stack too?

    started = False             # Has start() been called?
    processed = False           # Has begin() been called since last becoming top?

    def __init__(self, name=None):
        self.name = name

    def start(self):
        """Called ONCE when state is first instantiated and pushed."""
        pass

    def begin(self):
        """Called EVERY TIME this state becomes the top of the stack."""
        pass

    def take_input(self, event):
        """Process a single input event."""
        pass

    def update(self):
        """Per-frame game logic."""
        pass

```

```
    def draw(self, surf):
        """Render onto the given surface."""
        return surf

    def end(self):
        """Called when state is about to lose top-of-stack status."""
        pass

    def finish(self):
        """Called when state is being fully removed from the stack."""
        pass
```

Key Design Points

- `start()` **vs** `begin()`: `start()` runs once; `begin()` runs every time the state resurfaces; if you push a menu on top of `FreeState`, then pop the menu, `FreeState.begin()` runs again to restore the cursor and highlights—`start()` does not
- `end()` **vs** `finish()`: `end()` fires when the state is about to be covered or popped; `finish()` fires when the state object is actually removed from the stack
- **The** `'repeat'` **protocol**: any lifecycle method can return `'repeat'` to tell the state machine to skip the rest of the frame and re-run immediately; this is how single-frame states work

`MapState` extends `State` with map-specific rendering. Most in-game states extend `MapState` because they need the map drawn behind them:

```
class MapState(State):
    def __init__(self, name=None):
        if name:
            self.name = name
        self.fluid = FluidScroll()  # Smooth directional input handling

    def draw(self, surf, culled_rect=None):
        game.camera.update()
        game.highlight.update()
        camera_cull = (int(game.camera.get_x() * TILEWIDTH),
                       int(game.camera.get_y() * TILEHEIGHT),
                       WINWIDTH, WINHEIGHT)
        map_surf = game.map_view.draw(camera_cull, culled_rect)
        surf.blit(map_surf, (0, 0))
        return surf
```

The State Machine

The `StateMachine` manages the stack and drives the lifecycle:

```
class StateMachine():
    def __init__(self):
        self.state: List[State] = []        # The stack
        self.temp_state: List[str] = []     # Queued transitions (deferred)
        self.prev_state: State = None
        self.prior_state: State = None

    def change(self, new_state):
        """Queue a push. Doesn't execute until end of frame."""
        self.temp_state.append(new_state)

    def back(self):
```

```
        """Queue a pop."""
        self.temp_state.append('pop')

    def clear(self):
        """Queue clearing the entire stack."""
        self.temp_state.append('clear')

    def refresh(self):
        """Keep only the top state, discard everything below."""
        self.state = self.state[-1:]

    def current(self):
        """Name of the top state."""
        return self.state[-1].name if self.state else None

    def current_state(self) -> State:
        """The top state object."""
        return self.state[-1] if self.state else None
```

The critical method is `process_temp_state()`, which executes all queued transitions at the end of each frame:

```
def process_temp_state(self):
    for transition in self.temp_state:
        if transition == 'pop':
            state = self.state[-1]
            self.exit_state(state)     # Calls end() + finish()
            self.prior_state = state
            self.state.pop()
        elif transition == 'clear':
            self.prior_state = self.current_state()
            for state in reversed(self.state):
                self.exit_state(state)
            self.state.clear()
        else:
            # Instantiate new state from registry
            new_state = self.all_states[transition](transition)
            self.prior_state = self.state[-1] if self.state else None
            self.state.append(new_state)
    self.temp_state.clear()
```

State Lifecycle

Here is the frame loop—the `update()` method that runs every frame:

```
def update(self, event, surf):
    state = self.state[-1]
    repeat_flag = False

    # Phase 1: START (once per state instance)
    if not state.started:
        state.started = True
        if state.start() == 'repeat':
            repeat_flag = True

    # Phase 2: BEGIN (each time state becomes top)
    if not repeat_flag and not state.processed:
        state.processed = True
```

```
        if state.begin() == 'repeat':
            repeat_flag = True

    # Phase 3: TAKE INPUT
    if not repeat_flag:
        if state.take_input(event) == 'repeat':
            repeat_flag = True

    # Phase 4: UPDATE
    if not repeat_flag:
        if state.update() == 'repeat':
            repeat_flag = True

    # Phase 5: DRAW (with transparency walk)
    if not repeat_flag:
        idx = -1
        while self.state[idx].transparent and len(self.state) >= (abs(idx) + 1):
            idx -= 1
        while idx <= -1:
            surf = self.state[idx].draw(surf)
            idx += 1

    # Phase 6: END (if transitions are pending)
    if self.temp_state and state.processed:
        state.processed = False
        state.end()

    # Phase 7: Execute queued transitions
    self.process_temp_state()
    return surf, repeat_flag
```

Frame Loop Summary

```
+-------------------------------------------------+
|  Frame N                                        |
|                                                 |
|  1. START      (if first frame for this state)  |
|  2. BEGIN      (if state just became top)       |
|  3. TAKE_INPUT (process player input)           |
|  4. UPDATE     (game logic)                     |
|  5. DRAW       (render, walk transparency)      |
|  6. END        (if transitions queued)          |
|  7. TRANSITIONS (push / pop / clear)            |
|                                                 |
|  Any phase can return 'repeat' to skip the rest |
|  and re-run the state machine this frame.       |
+-------------------------------------------------+
```

The `'repeat'` mechanism is crucial for states that do all their work in `start()` or `begin()`—they set up transitions and immediately return `'repeat'` so the new state starts in the same frame. Without this, you would see blank frames during rapid transitions.

Transparency and Layered Rendering

The `transparent` flag enables visual layering without coupling states together:

```
State Stack (top to bottom):
+------------------+
```

```
|  EventState      |   transparent = True  <- draws dialog overlay
+------------------+
|  MenuState       |   transparent = True  <- draws action menu
+------------------+
|  FreeState       |   transparent = False <- draws map + units
+------------------+

Drawing order (bottom to top):
1. FreeState.draw()  -> map, units, cursor
2. MenuState.draw()  -> action menu overlay
3. EventState.draw() -> dialog box overlay

Result: Dialog on top of menu on top of map. Clean compositing.
```

The state machine walks *down* the stack until it finds a non-transparent state, then draws *up*. Each state only draws its own layer—it does not need to know what is above or below it.

Inter-State Communication

States communicate through `game.memory`, a shared dictionary on the `GameState` singleton. This is a **message bus pattern**—the sender writes to a known key, the receiver reads it; neither state imports the other:

```python
# In WeaponChoiceState, after the player picks a weapon:
game.memory['item'] = selected_weapon
game.state.change('combat_targeting')

# In CombatTargetingState.start():
self.item = game.memory['item']
```

Common patterns:

```python
# Pass data forward
game.memory['item'] = weapon
game.memory['current_unit'] = unit
game.memory['next_state'] = 'info_menu'

# TransitionToState reads 'next_state' and navigates there
class TransitionToState(TransitionOutState):
    def update(self):
        if engine.get_time() >= self.start_time + self.wait_time:
            game.state.back()
            game.state.change(game.memory['next_state'])
            return 'repeat'
```

The convention is that the state which writes a key is responsible for what goes in it. `game.memory` is intentionally untyped—it trades compile-time safety for zero coupling.

The State Registry

States are registered by name in `StateMachine.load_states()`:

```python
def load_states(self, starting_states=None, temp_state=None):
    self.all_states = {
        # Title
```

```
        'title_start':      title_screen.TitleStartState,
        'title_main':       title_screen.TitleMainState,
        'title_load':       title_screen.TitleLoadState,

        # Transitions
        'transition_in':    transitions.TransitionInState,
        'transition_out':   transitions.TransitionOutState,
        'transition_to':    transitions.TransitionToState,

        # Core gameplay
        'free':             general_states.FreeState,
        'move':             general_states.MoveState,
        'movement':         general_states.MovementState,
        'menu':             general_states.MenuState,
        'wait':             general_states.WaitState,

        # Combat
        'weapon_choice':    general_states.WeaponChoiceState,
        'combat_targeting': general_states.CombatTargetingState,
        'combat':           general_states.CombatState,
        'dying':            general_states.DyingState,

        # AI
        'ai':               general_states.AIState,

        # Events
        'event':            event_state.EventState,

        # Phase management
        'turn_change':      general_states.TurnChangeState,
        'phase_change':     general_states.PhaseChangeState,

        # ... 100+ more states
    }
```

State navigation always uses string names. States never reference each other's classes directly—adding a new state is two steps: write the class, add one line to the registry:

```
game.state.change('combat')   # Push by name
game.state.back()             # Pop (no name needed)
game.state.clear()            # Clear all
```

Serialization

Saving game state is remarkably simple because the stack is just a list of names:

```
def save(self):
    return ([state.name for state in self.state],
            self.temp_state[:])  # Copy to avoid mutation
```

A save file might contain:

```
state_stack = ['free', 'move', 'menu']
temp_state = []
```

Loading reconstructs the stack by re-instantiating from the registry. Individual states re-derive their display state from game data (cursor position, unit selection, etc.) in their `start()` and `begin()` methods. This is why `begin()` exists separately from `start()`—it is the "reconstruct yourself from current game state" hook.

Concrete State Examples

Single-Frame States

Some states exist for exactly one frame: they do work, queue a transition, and immediately return `'repeat'`.

WaitState—marks all attacked units as finished:

```
class WaitState(MapState):
    name = 'wait'

    def update(self):
        super().update()
        game.state.back()
        for unit in game.units:
            if unit.has_attacked and not unit.finished:
                unit.wait()
        return 'repeat'
```

This is a *command* disguised as a state. It exists because the state machine is the only sequencing mechanism—if you need "do X then do Y," you push state Y, then push state X. X runs, pops itself, and Y naturally becomes top.

TurnChangeState—orchestrates the turn/phase transition:

```
class TurnChangeState(MapState):
    name = 'turn_change'

    def begin(self):
        game.state.refresh()  # Clear the stack except me
        game.state.back()     # Pop myself too
        return 'repeat'

    def end(self):
        game.phase.next()
        if game.phase.get_current() == 'player':
            # Stack: free -> status_upkeep -> phase_change (top)
            # Executes right-to-left: phase_change, then status_upkeep, then free
            game.state.change('free')
            game.state.change('status_upkeep')
            game.state.change('phase_change')
            game.events.trigger(triggers.TurnChange())
        else:
            game.state.change('ai')
            game.state.change('status_upkeep')
            game.state.change('phase_change')
```

Reading Push Order

States are pushed in the order `free`, `status_upkeep`, `phase_change`. Since it is a stack, `phase_change` runs first, pops, then `status_upkeep`, pops, then `free` becomes active. **Read the pushes bottom-to-top**

for execution order.

Interactive States

FreeState—the main player turn state, where the player selects units and issues commands:

```
class FreeState(MapState):
    name = 'free'

    def begin(self):
        game.cursor.show()
        game.boundary.show()
        phase.fade_in_phase_music()

        # Auto-end turn if all units are done
        if all(u.finished for u in game.get_player_units()
               if skill_system.can_select(u)):
            game.state.change('turn_change')
            game.state.change('status_endstep')
            game.state.change('ai')
            return 'repeat'

    def take_input(self, event):
        game.cursor.take_input()
        if event == 'SELECT':
            cur_unit = game.board.get_unit(game.cursor.position)
            if cur_unit and not cur_unit.finished \
                    and skill_system.can_select(cur_unit):
                game.cursor.cur_unit = cur_unit
                game.state.change('move')  # Push MoveState
            else:
                game.state.change('option_menu')
        elif event == 'INFO':
            if game.cursor.get_hover():
                game.memory['current_unit'] = game.cursor.get_hover()
                game.memory['next_state'] = 'info_menu'
                game.state.change('transition_to')

    def update(self):
        super().update()
        game.highlight.handle_hover()

    def end(self):
        game.cursor.set_speed_state(False)
        game.highlight.remove_highlights()
```

`FreeState.begin()` demonstrates the auto-end-turn pattern: if all player units have acted, it does not wait for input—it immediately queues the turn-change sequence and returns `'repeat'`.

MoveState—handles unit movement and transitions to the action menu:

```
class MoveState(MapState):
    name = 'move'

    def begin(self):
        cur_unit = game.cursor.cur_unit
        cur_unit.sprite.change_state('selected')
        self.valid_moves = game.highlight.display_highlights(cur_unit)
        game.cursor.show_arrows()

```

```
    def take_input(self, event):
        cur_unit = game.cursor.cur_unit
        if event == 'SELECT':
            if game.cursor.position in self.valid_moves:
                cur_unit.current_move = action.Move(
                    cur_unit, game.cursor.position)
                game.state.change('menu')      # Menu runs after movement
                game.state.change('movement')  # Movement animation runs first
                action.do(cur_unit.current_move)
        elif event == 'BACK':
            game.cursor.set_pos(cur_unit.position)
            game.state.clear()
            game.state.change('free')

    def end(self):
        game.cursor.remove_arrows()
        game.highlight.remove_highlights()
```

The push ordering `change('menu')` then `change('movement')` means `MovementState` (the animation) runs first, and when it pops, `MenuState` becomes active. The state machine is your sequencer.

Orchestrator States

CombatState—manages the entire combat encounter:

```
class CombatState(MapState):
    name = 'combat'

    def start(self):
        game.cursor.hide()
        self.combat = game.combat_instance.pop(0)
        game.memory['current_combat'] = self.combat
        self.is_animation_combat = isinstance(
            self.combat, interaction.AnimationCombat)

    def take_input(self, event):
        if event == 'START':
            self.combat.skip()  # Skip animation

    def update(self):
        super().update()
        done = self.combat.update()  # Delegates to combat system
        if done:
            return 'repeat'

    def draw(self, surf):
        if self.is_animation_combat and self.combat.viewbox:
            surf = super().draw(surf, culled_rect=self.combat.viewbox)
            surf.blit(self.fuzz_background, (0, 0))
        else:
            surf = super().draw(surf)
        self.combat.draw(surf)
        return surf
```

Notice that `CombatState` does not *implement* combat—it delegates to `self.combat`, which is an `AnimationCombat` or `BaseCombat` instance. The state manages the lifecycle (start, skip, done-check) and rendering integration only.

EventState—runs scripted events and cutscenes:

```
class EventState(State):
    name = 'event'
    transparent = True  # Map/game continues rendering underneath

    def begin(self):
        if not self.event:
            self.event = game.events.get()
            game.cursor.hide()

    def update(self):
        if self.event:
            self.event.update()
        else:
            game.state.back()  # No more events, pop
            return 'repeat'
        if self.event.state == 'complete':
            return self.end_event()

    def draw(self, surf):
        if self.event:
            self.event.draw(surf)
        return surf

    def end_event(self):
        game.events.end(self.event)
        if game.level_vars.get('_win_game'):
            self.level_end()
        elif game.level_vars.get('_lose_game'):
            game.memory['next_state'] = 'game_over'
            game.state.change('transition_to')
```

`EventState` is `transparent = True`, so the map draws behind the dialog. When the event completes, it checks win/loss conditions and routes accordingly.

Transition States

Transitions are small, elegant states that handle screen fades:

```
class TransitionInState(State):
    """Fade from black to gameplay."""
    name = 'transition_in'
    transparent = True

    def start(self):
        self.bg = SPRITES.get('bg_black').convert_alpha()
        self.start_time = engine.get_time()
        self.wait_time = game.memory.get('transition_speed', 1) * 133

    def update(self):
        if engine.get_time() >= self.start_time + self.wait_time:
            game.state.back()
            return 'repeat'

    def draw(self, surf):
        proc = (engine.get_time() - self.start_time) / self.wait_time
        bg = image_mods.make_translucent(self.bg, proc)
        engine.blit_center(surf, bg)
        return surf

```

```
22
23 class TransitionToState(TransitionOutState):
24     """Fade to black, then navigate to game.memory['next_state']."""
25     name = 'transition_to'
26
27     def update(self):
28         if engine.get_time() >= self.start_time + self.wait_time:
29             game.state.back()
30             game.state.change(game.memory['next_state'])
31             return 'repeat'
```

`TransitionToState` is the standard "navigate with a fade" pattern: the caller writes the destination to `game.memory['next_state']`, pushes `transition_to`, and the transition handles the rest. Variants include `TransitionPopState` (fade then pop) and `TransitionDoublePopState` (fade then pop twice) for different navigation needs.

How It Is Used and Extended

Adding a New State

1. **Write the class**:

```
1  from app.engine.state import MapState
2  from app.engine.game_state import game
3
4  class MyFeatureState(MapState):
5      name = 'my_feature'
6      transparent = True  # If you want the map visible behind it
7
8      def start(self):
9          self.data = game.memory.get('feature_data')
10
11     def take_input(self, event):
12         if event == 'BACK':
13             game.state.back()
14             return 'repeat'
15         elif event == 'SELECT':
16             game.state.back()
17
18     def draw(self, surf):
19         surf = super().draw(surf)
20         # Draw your feature's UI on top
21         return surf
```

2. **Register it** (one line in `state_machine.py`):

```
1  self.all_states['my_feature'] = my_feature.MyFeatureState
```

3. **Navigate to it** from any other state:

```
1  game.memory['feature_data'] = some_data
2  game.state.change('my_feature')
```

Patterns Worth Knowing

Stack sequencing—push in reverse execution order:

```
# Execute: phase_change -> status_upkeep -> free
game.state.change('free')           # Will run third
game.state.change('status_upkeep')  # Will run second
game.state.change('phase_change')   # Will run first (it's on top)
```

Transition wrapping—use `transition_to` for visual navigation:

```
game.memory['next_state'] = 'info_menu'
game.state.change('transition_to')  # Fades out, navigates, fades in
```

Single-frame command states—do work and pop immediately:

```
class DoSomethingState(MapState):
    name = 'do_something'

    def begin(self):
        do_the_thing()
        game.state.back()
        return 'repeat'  # Skip rendering, continue immediately
```

Adapting to Godot

Godot has its own scene and node tree, but the pushdown state machine pattern applies directly. Here is the port.

Step 1: The Base State

```
class_name GameState extends RefCounted

var state_name: String = ""
var transparent: bool = false
var started: bool = false
var processed: bool = false

func start() -> String:
    return ""

func begin() -> String:
    return ""

func take_input(event: InputEvent) -> String:
    return ""

func update(delta: float) -> String:
    return ""

func draw(canvas: CanvasItem) -> void:
    pass

```

```
func end() -> void:
    pass

func finish() -> void:
    pass
```

Step 2: The State Machine

```
class_name GameStateMachine extends Node

var stack: Array[GameState] = []
var pending: Array = []  # Array of String, "pop", or "clear"
var state_registry: Dictionary = {}
var memory: Dictionary = {}

func register_state(name: String, state_class: GDScript) -> void:
    state_registry[name] = state_class

func change(state_name: String) -> void:
    pending.append(state_name)

func back() -> void:
    pending.append("pop")

func clear() -> void:
    pending.append("clear")

func current() -> GameState:
    return stack.back() if stack.size() > 0 else null

func _process(delta: float) -> void:
    if stack.is_empty():
        return

    var state := stack.back()
    var repeat := false

    # START
    if not state.started:
        state.started = true
        if state.start() == "repeat":
            repeat = true

    # BEGIN
    if not repeat and not state.processed:
        state.processed = true
        if state.begin() == "repeat":
            repeat = true

    # INPUT
    if not repeat:
        var event := _get_current_input()
        if state.take_input(event) == "repeat":
            repeat = true

    # UPDATE
    if not repeat:
        if state.update(delta) == "repeat":
            repeat = true

    # DRAW (handled in _draw or CanvasLayer)
```

```
    if not repeat:
        _draw_stack()

    # END
    if not pending.is_empty() and state.processed:
        state.processed = false
        state.end()

    # PROCESS TRANSITIONS
    _process_pending()

func _process_pending() -> void:
    for transition: Variant in pending:
        if transition == "pop":
            if stack.size() > 0:
                var s: GameState = stack.pop_back()
                s.end()
                s.finish()
        elif transition == "clear":
            while stack.size() > 0:
                var s: GameState = stack.pop_back()
                s.end()
                s.finish()
        else:
            var new_state: GameState = state_registry[transition].new()
            new_state.state_name = transition
            stack.append(new_state)
    pending.clear()

func _draw_stack() -> void:
    # Walk down to find first non-transparent state
    var start_idx := stack.size() - 1
    while start_idx > 0 and stack[start_idx].transparent:
        start_idx -= 1
    # Draw upward from first opaque state
    for i: int in range(start_idx, stack.size()):
        stack[i].draw(get_tree().root)
```

Step 3: Concrete States

```
class_name FreeState extends GameState

func _init() -> void:
    state_name = "free"

func begin() -> String:
    GameManager.cursor.show()
    GameManager.highlight.show_boundary()
    return ""

func take_input(event: InputEvent) -> String:
    if event.is_action_pressed("select"):
        var unit = GameManager.board.get_unit(GameManager.cursor.position)
        if unit and not unit.finished:
            GameManager.memory["selected_unit"] = unit
            GameManager.state_machine.change("move")
    return ""

func update(_delta: float) -> String:
    GameManager.highlight.handle_hover()
    return ""
```

```
func end() -> void:
    GameManager.highlight.clear()
```

```
class_name CombatState extends GameState

var combat_instance: Variant

func _init() -> void:
    state_name = "combat"

func start() -> String:
    combat_instance = GameManager.memory["combat"]
    GameManager.cursor.hide()
    return ""

func update(delta: float) -> String:
    if combat_instance.update(delta):
        GameManager.state_machine.back()
        return "repeat"
    return ""

func draw(canvas: CanvasItem) -> void:
    combat_instance.draw(canvas)
```

Step 4: Per-Entity State Machines

The pattern also works *per-entity* for AI and animation states. Give each unit its own mini state machine:

```
class_name UnitStateMachine extends Node

var stack: Array = []

class UnitState:
    func enter(_unit: Node) -> void: pass
    func update(_unit: Node, _delta: float) -> String: return ""
    func exit(_unit: Node) -> void: pass

class IdleState extends UnitState:
    func enter(unit: Node) -> void:
        unit.play_animation("idle")

class MovingState extends UnitState:
    var target_pos: Vector2i
    func enter(unit: Node) -> void:
        unit.play_animation("walk")
    func update(unit: Node, _delta: float) -> String:
        if unit.position == target_pos:
            return "done"
        return ""

class AttackingState extends UnitState:
    func enter(unit: Node) -> void:
        unit.play_animation("attack")
    func update(unit: Node, _delta: float) -> String:
        if unit.animation_finished:
            return "done"
        return ""
```

This gives entities their own behavioral state machines independent of the global game state machine. The global machine handles *game flow* ("whose turn is it?"), while per-entity machines handle *entity behavior* ("is this unit idle, moving, or attacking?").

Architecture Comparison

Lex Talionis (Python)	Godot 4 (GDScript)
`State` base class	`GameState extends RefCounted`
`MapState` (map-rendering state)	`MapGameState` (draws tilemap)
`StateMachine.state` (list)	`GameStateMachine.stack` (Array)
`StateMachine.temp_state`	`GameStateMachine.pending`
`game.state.change('name')`	`state_machine.change("name")`
`game.state.back()`	`state_machine.back()`
`game.memory` (dict)	`state_machine.memory` (Dictionary)
`all_states` (str → class dict)	`state_registry` (str → class dict)
`transparent = True`	`transparent = true`
`'repeat'` return value	`"repeat"` return value
`State.start` / `begin` / `end`	Same lifecycle methods

Conclusion

The Lex Talionis State System is a textbook implementation of the **pushdown automaton** applied to game architecture—and the fact that it manages 120+ states without collapsing into chaos is proof that the pattern works at scale.

Key Takeaways

1. **Stack-based state machines** naturally model the nested, interruptible flow that SRPGs and other complex games require; flat FSMs cannot cut it
2. **Deferred transitions** (queuing changes instead of executing immediately) prevent re-entrancy bugs and ensure consistent per-frame behavior
3. **The `'repeat'` protocol** lets single-frame states exist—states that set up transitions and skip rendering, acting as sequencing commands rather than visual modes
4. **Transparency** decouples visual layering from state logic; a menu state does not need to know how to draw the map—it just declares `transparent = True` and the machine composites them
5. `game.memory` provides loose coupling between states; states communicate through a shared bus, not through imports or method calls
6. **String-based registration** means adding a new state is two steps: write the class, add one line to the registry; no switch statements, no enum updates, no base class modifications

The pattern is universal. Whether you are building in Pygame, Godot, Unity, or a custom C++ engine, a stack-based state machine with deferred transitions and transparent layering is one of those patterns that, once you have used it, you wonder how you ever built games without it.

This document references our private dev fork of the Lex Talionis engine here, where I work together with some other contributors to produce a fork of the engine suitable for deployment on Steam and other commercial platforms. All code snippets are drawn from the codebase and lightly adapted for clarity. You may view the master branch of the Lex Talionis engine at my personal mirror or at the source.